三次样条插值方法上机报告

周才翔
数试2402 2243711412

2026 年 5 月 5 日

目录

1 引言

在科学计算与工程应用中，函数插值是逼近未知函数的核心方法。高次多项式插值存在龙格现象和数值不稳定问题，而分段线性、分段二次等低次插值虽避免了上述缺陷，但在节点处导数不连续，无法满足飞机机翼设计、船体放样、数控加工等对曲线二阶光滑性的要求。

三次样条插值方法解决了这一矛盾。它通过在每个子区间构造三次多项式，同时保证整个区间上二阶导数连续，兼具精度高、稳定性好、光滑性优的特点，是目前工程中应用广泛的插值方法。

2 三次样条插值方法原理

2.1 基本定义

设区间 $[a,b]$ 上有 $n+1$ 个互异节点 $a=x_0<x_1<\cdots<x_n=b$，对应函数值 $y_i=f(x_i)$ $(i=0,1,\cdots,n)$。若函数 $S(x)$ 满足以下三个条件，则称其为 $f(x)$ 在 $[a,b]$ 上的三次样条插值函数：

1. 在每个子区间 $[x_{i-1},x_i]$ $(i=1,2,\cdots,n)$ 上，$S(x)$ 是三次多项式；

2. 在所有节点上满足 $S(x_i)=y_i$ $(i=0,1,\cdots,n)$；

3. 在整个区间 $[a,b]$ 上，$S(x)$ 的二阶导数连续，即 $S''(x)\in C[a,b]$。

2.2 三弯矩方程组推导

设 $M_i=S''(x_i)$ 为节点 x_i 处的二阶导数值（力学中称为弯矩）。由于 $S(x)$ 在子区间 $[x_{i-1},x_i]$ 上是三次多项式，其二阶导数为线性函数：

$$S''(x)=\frac{x_i-x}{h_i}M_{i-1}+\frac{x-x_{i-1}}{h_i}M_i,\quad x\in[x_{i-1},x_i]$$

其中 $h_i=x_i-x_{i-1}$ 为子区间步长。

对 $S''(x)$ 积分两次，利用插值条件 $S(x_{i-1})=y_{i-1}$、$S(x_i)=y_i$ 确定积分常数，得到子区间上的样条表达式：

$$S(x)=\frac{M_{i-1}(x_i-x)^3}{6h_i}+\frac{M_i(x-x_{i-1})^3}{6h_i}+\left(y_{i-1}-\frac{M_{i-1}h_i^2}{6}\right)\frac{x_i-x}{h_i}+\left(y_i-\frac{M_ih_i^2}{6}\right)\frac{x-x_{i-1}}{h_i}$$

利用一阶导数在内部节点连续的条件 $S'_-(x_i)=S'_+(x_i)$ $(i=1,2,\cdots,n-1)$，推导得到关于 M_i 的三弯矩方程组：

$$\mu_iM_{i-1}+2M_i+\lambda_iM_{i+1}=d_i,\quad i=1,2,\cdots,n-1$$

其中：

- 权重系数：$\mu_i = \frac{h_i}{h_i+h_{i+1}}, \quad \lambda_i = 1-\mu_i$
- 右端项：$d_i = \frac{6}{h_i+h_{i+1}} \cdot \left(\frac{y_{i+1}-y_i}{h_{i+1}} - \frac{y_i-y_{i-1}}{h_i}\right)$

2.3 三种标准边界条件

三弯矩方程组共 $n-1$ 个方程，未知数有 $n+1$ 个，需补充2个边界条件唯一确定解：

1. 二阶导数边界：已知两端点的二阶导数值 $f''(a)$ 和 $f''(b)$，即 $M_0 = f''(a),\ M_n = f''(b)$。特别地，当 $M_0 = M_n = 0$ 时，称为自然三次样条。

2. 一阶导数边界：已知两端点的一阶导数值 $f'(a)$ 和 $f'(b)$，对应边界方程：

$$2M_0 + M_1 = \frac{6}{h_1}\left(\frac{y_1-y_0}{h_1} - f'(a)\right), \quad M_{n-1} + 2M_n = \frac{6}{h_n}\left(f'(b) - \frac{y_n-y_{n-1}}{h_n}\right)$$

3. 周期边界：函数以 $T = b-a$ 为周期，满足 $y_0 = y_n$，且 $M_0 = M_n$，对应边界方程：

$$\mu_n M_{n-1} + 2M_n + \lambda_n M_1 = d_n, \quad \mu_n = \frac{h_n}{h_n+h_1},\ \lambda_n = \frac{h_1}{h_n+h_1}$$

2.4 算法流程

1.计算h_i
2.计算μ_i, λ_i
3.计算d_i
4.根据边界条件解对应的线性方程组
5.计算方程$S(x)$

3 Python编程实现

```
class CubicSpline:
    def __init__(self, x, y, boundary_type, m0=0, mn=0, yp0=0, ypn=0):
        self.x = x
        self.y = y
        self.n = len(x) - 1
        self.h = [x[i+1]-x[i] for i in range(self.n)]

        size = self.n + 1
        matrix = [[0.0]*(size+1) for _ in range(size)]
```

```
for i in range(1, self.n):
    mu = self.h[i-1] / (self.h[i-1]+self.h[i])
    Lambda = self.h[i] / (self.h[i-1]+self.h[i])
    d = 6 * ((y[i+1]-y[i])/self.h[i] - (y[i]-y[i-1])/self.h[i-1])
    / (self.h[i-1]+self.h[i])
    matrix[i][i-1] = mu
    matrix[i][i] = 2.0
    matrix[i][i+1] = Lambda
    matrix[i][-1] = d

if boundary_type == 1: # 二阶导
    matrix[0][0] = 1.0
    matrix[0][-1] = m0
    matrix[self.n][self.n] = 1.0
    matrix[self.n][-1] = mn
elif boundary_type == 2: # 一阶导
    matrix[0][0] = 2.0
    matrix[0][1] = 1.0
    matrix[0][-1] = 6 * ((y[1]-y[0])/self.h[0] - yp0) / self.h[0]
    matrix[self.n][self.n-1] = 1.0
    matrix[self.n][self.n] = 2.0
    matrix[self.n][-1] = 6 * (ypn - (y[self.n]-y[self.n-1])
    /self.h[self.n-1]) / self.h[self.n-1]
elif boundary_type == 3: # 周期
    size_p = self.n
    matrix = [[0.0]*(size_p+1) for _ in range(size_p)]

    for i in range(1, size_p):
        mu = self.h[i-1] / (self.h[i-1]+self.h[i])
        Lambda = self.h[i] / (self.h[i-1]+self.h[i])
        d = 6 * ((y[i+1]-y[i])/self.h[i] - (y[i]-y[i-1])
        /self.h[i-1]) / (self.h[i-1]+self.h[i])
        matrix[i][i-1] = mu
        matrix[i][i] = 2.0
        matrix[i][i+1] = Lambda
        matrix[i][-1] = d
```

```
            mu = self.h[-1] / (self.h[-1]+self.h[0])
            Lambda = self.h[0] / (self.h[-1]+self.h[0])
            d = 6 * ((y[1]-y[0])/self.h[0] - (y[-1]-y[-2])/self.h[-1])
            / (self.h[-1]+self.h[0])
            matrix[0][0] = 2.0
            matrix[0][1] = Lambda
            matrix[0][-1] = d
            matrix[0][-2] = mu

            i = size_p - 1
            mu = self.h[i-1] / (self.h[i-1]+self.h[i])
            Lambda = self.h[i] / (self.h[i-1]+self.h[i])
            d = 6 * ((y[i+1]-y[i])/self.h[i] - (y[i]-y[i-1])/self.h[i-1])
            / (self.h[i-1]+self.h[i])
            matrix[i][i-1] = mu
            matrix[i][i] = 2.0
            matrix[i][-1] = d
            matrix[i][0] = Lambda

            self.M = self._gauss_eliminate(matrix)
            self.M.append(self.M[0])
            return

        self.M = self._gauss_eliminate(matrix)

    def _gauss_eliminate(self, matrix):
        n = len(matrix)
        for i in range(n):

            max_row = i
            for r in range(i, n):
                if abs(matrix[r][i]) > abs(matrix[max_row][i]):
                    max_row = r
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

            zhuyuan = matrix[i][i]
            if abs(zhuyuan) < 1e-12: break
```

```
        for j in range(i, n+1):
            matrix[i][j] /= zhuyuan

        for r in range(n):
            if r != i and abs(matrix[r][i]) > 1e-12:
                factor = matrix[r][i]
                for j in range(i, n+1):
                    matrix[r][j] -= factor * matrix[i][j]
    return [row[-1] for row in matrix]

def evaluate(self, x_val):
    k = 0
    for i in range(self.n):
        if self.x[i] <= x_val <= self.x[i+1]:
            k = i
            break
    if x_val == self.x[-1]: k = self.n-1

    hk = self.h[k]
    t1 = self.M[k] * (self.x[k+1]-x_val)**3 / (6*hk)
    t2 = self.M[k+1] * (x_val-self.x[k])**3 / (6*hk)
    t3 = (self.y[k] - self.M[k]*hk**2/6) * (self.x[k+1]-x_val) / hk
    t4 = (self.y[k+1] - self.M[k+1]*hk**2/6) * (x_val-self.x[k]) / hk
    return t1 + t2 + t3 + t4

def get_equation(self):
    equations = []
    for k in range(self.n):
        xk = self.x[k]
        xk1 = self.x[k+1]
        yk = self.y[k]
        yk1 = self.y[k+1]
        Mk = self.M[k]
        Mk1 = self.M[k+1]
        hk = self.h[k]

        c1 = Mk / (6 * hk)
```

```
c2 = Mk1 / (6 * hk)
c3 = (yk - Mk * hk**2 / 6) / hk
c4 = (yk1 - Mk1 * hk**2 / 6) / hk

A = -c1 + c2
B = 3*c1*xk1 - 3*c2*xk
C = -3*c1*xk1**2 + 3*c2*xk**2 - c3 + c4
D = c1*xk1**3 - c2*xk**3 + c3*xk1 + c4*(-xk)

def format_term(coef, power, is_first=False):
    if abs(coef) < 1e-8:
        return ""
    if is_first:
        sign = "-" if coef < 0 else ""
    else:
        sign = " - " if coef < 0 else " + "
    c_abs = abs(coef)
    if c_abs == int(c_abs):
        c_str = f"{int(c_abs)}"
    else:
        c_str = f"{c_abs:.4f}"

    if power == 3:
        x_str = "x^3"
    elif power == 2:
        x_str = "x^2"
    elif power == 1:
        x_str = "x"
    else:
        x_str = ""

    if c_abs == 1 and power > 0:
        return f"{sign}{x_str}"
    else:
        return f"{sign}{c_str}{x_str}"

eq_str = ""
```

```
            parts = [format_term(A, 3, True), format_term(B, 2),
            format_term(C, 1), format_term(D, 0)]
            eq_str = "".join([p for p in parts if p])

            if not eq_str:
                eq_str = "0"

            interval_str = f"x ∈ [{xk}, {xk1}]"
            equations.append((eq_str, interval_str))

        return equations

def main():
    print("三次样条插值法计算器")

    try:
        n_points = int(input("请输入插值节点的个数 (n+1): "))
        x_list = []
        y_list = []
        print(f"\n请依次输入 {n_points} 个节点数据 (x y), 用空格分隔: ")
        for i in range(n_points):
            while True:
                try:
                    line = input(f"  第 {i+1} 个点: ")
                    x, y = map(float, line.split())
                    x_list.append(x)
                    y_list.append(y)
                    break
                except:
                    print("输入格式错误, 请重新输入 (例如: 0 1)")

        if x_list != sorted(x_list):
            pairs = sorted(zip(x_list, y_list))
            x_list, y_list = zip(*pairs)
            x_list = list(x_list)
            y_list = list(y_list)
```

```
except ValueError:
    print("输入错误，请输入整数")
    return

print("\n" + "-"*30)
print("请选择边界条件类型：")
print("1. 二阶导数边界")
print("2. 一阶导数边界")
print("3. 周期边界")

choice = 0
while choice not in [1, 2, 3]:
    try:
        choice = int(input("请输入选项 (1/2/3)："))
    except:
        pass

m0, mn, yp0, ypn = 0, 0, 0, 0
if choice == 1:
    print("\n[二阶导数边界]")
    m0 = float(input(f"请输入 S''({x_list[0]}) = "))
    mn = float(input(f"请输入 S''({x_list[-1]}) = "))
elif choice == 2:
    print("\n[一阶导数边界]")
    yp0 = float(input(f"请输入 S'({x_list[0]}) = "))
    ypn = float(input(f"请输入 S'({x_list[-1]}) = "))
elif choice == 3:
    print("\n[周期边界]")

spline = CubicSpline(x_list, y_list, choice, m0, mn, yp0, ypn)

print("计算结果：")
print("节点弯矩 M_i:")
for i, m in enumerate(spline.M):
    print(f"  M[{i}] = {m:.6f}")

print("插值函数 S(x) 分段表达式：")
```

```
    eqs = spline.get_equation()
    for i, (eq_str, interval) in enumerate(eqs):
        print(f"  第 {i+1} 段: S(x) = {eq_str}")
        print(f"{interval}\n")

    print("现在可以输入任意x计算插值值 (输入 'q' 退出):")
    while True:
        cmd = input("x = ")
        if cmd.lower() == 'q':
            break
        try:
            xq = float(cmd)
            if xq < x_list[0] or xq > x_list[-1]:
                print(f"x={xq} 超出插值区间 [{x_list[0]}, {x_list[-1]}]")
            yq = spline.evaluate(xq)
            print(f"  S({xq}) = {yq:.6f}")
        except:
            print("输入无效，请输入数字或'q'")

if __name__ == "__main__":
    main()
```

4 实例计算结果

实例一：二阶导数边界（教材例4.6.1）

已知数据：

x_i	-3	-1	0	3	4
y_i	7	11	26	56	29

边界条件：二阶导数边界，$M_0 = 0,\ M_4 = 0$

计算结果：

节点弯矩：$M = [0.000000, 12.000000, 6.000000, -30.000000, 0.000000]$

分段插值表达式：

$$S(x) = \begin{cases} x^3 + 9x^2 + 25x + 28, & x \in [-3.0, -1.0] \\ -x^3 + 3x^2 + 19x + 26, & x \in [-1.0, 0.0] \\ -2x^3 + 3x^2 + 19x + 26, & x \in [0.0, 3.0] \\ 5x^3 - 60x^2 + 208x - 163, & x \in [3.0, 4.0] \end{cases}$$

实例二：一阶导数边界（教材习题4.14）

已知数据：

x_i	0.25	0.30	0.39	0.45	0.53
y_i	0.5000	0.5477	0.6245	0.6708	0.7280

边界条件：一阶导数边界，$S'(0.25) = 1.0000,\ S'(0.53) = 0.6868$

计算结果：

节点弯矩：$M = [-2.028630, -1.462741, -1.033345, -0.805830, -0.654585]$

分段插值表达式：

$$S(x) = \begin{cases} 1.8863x^3 - 2.4290x^2 + 1.8608x + 0.1571, & x \in [0.25, 0.30] \\ 0.7952x^3 - 1.4470x^2 + 1.5662x + 0.1866, & x \in [0.30, 0.39] \\ 0.6320x^3 - 1.2561x^2 + 1.4918x + 0.1963, & x \in [0.39, 0.45] \\ 0.3151x^3 - 0.8283x^2 + 1.2993x + 0.2251, & x \in [0.45, 0.53] \end{cases}$$

实例三：周期边界

已知数据：周期函数 $y = \cos(x)$ 在一个周期 $[0, 2\pi]$ 上的5个节点

x_i	0	$\pi/2$	π	$3\pi/2$	2π
y_i	1	0	-1	0	1

边界条件：周期边界

计算结果：

节点弯矩：$M = [-1.215849, 0.000000, 1.215849, 0.000000, -1.215849]$

分段插值表达式：

$$S(x) = \begin{cases} 0.1290x^3 - 0.6079x^2 + 1.0000, & x \in [0.0, 1.5708] \\ 0.1290x^3 - 0.6079x^2 + 1.0000x, & x \in [1.5708, 3.1416] \\ -0.1290x^3 + 1.8238x^2 - 7.6394x + 9.0000, & x \in [3.1416, 4.7124] \\ -0.1290x^3 + 1.8238x^2 - 7.6394x + 9.0000, & x \in [4.7124, 6.2832] \end{cases}$$

5 结论

本报告简述了三次样条插值法的数学原理，基于Python语言实现了支持三种标准边界条件的插值程序，并通过实例验证了算法的正确性。